DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING • PES UNIVERSITY, BENGALURU

UE24CS395: Database Management Systems Mini Project • Academic Year 2025–2026

Comprehensive Project Report & Master Evaluation Map

Evaluation Matrix Rows 10, 11, 13, 14: Project Report • Weightage: 11 Marks

Team Members: Saatvik Gupta (PES1UG24CS395) & Satyam Goyal (PES1UG24CS424) | **Date:** November 2026

1. Project Architecture & System Integration

The Intelligence Fusion Platform is an end-to-end case management system built around a polyglot persistence architecture. The platform connects a Next.js 15 client single-page application (SPA), an asynchronous FastAPI 3.13 backend, a 22-table PostgreSQL relational database (with automated SQLite fallback), and a MongoDB 7.0 document store with a 64-dimensional semantic vector search engine.

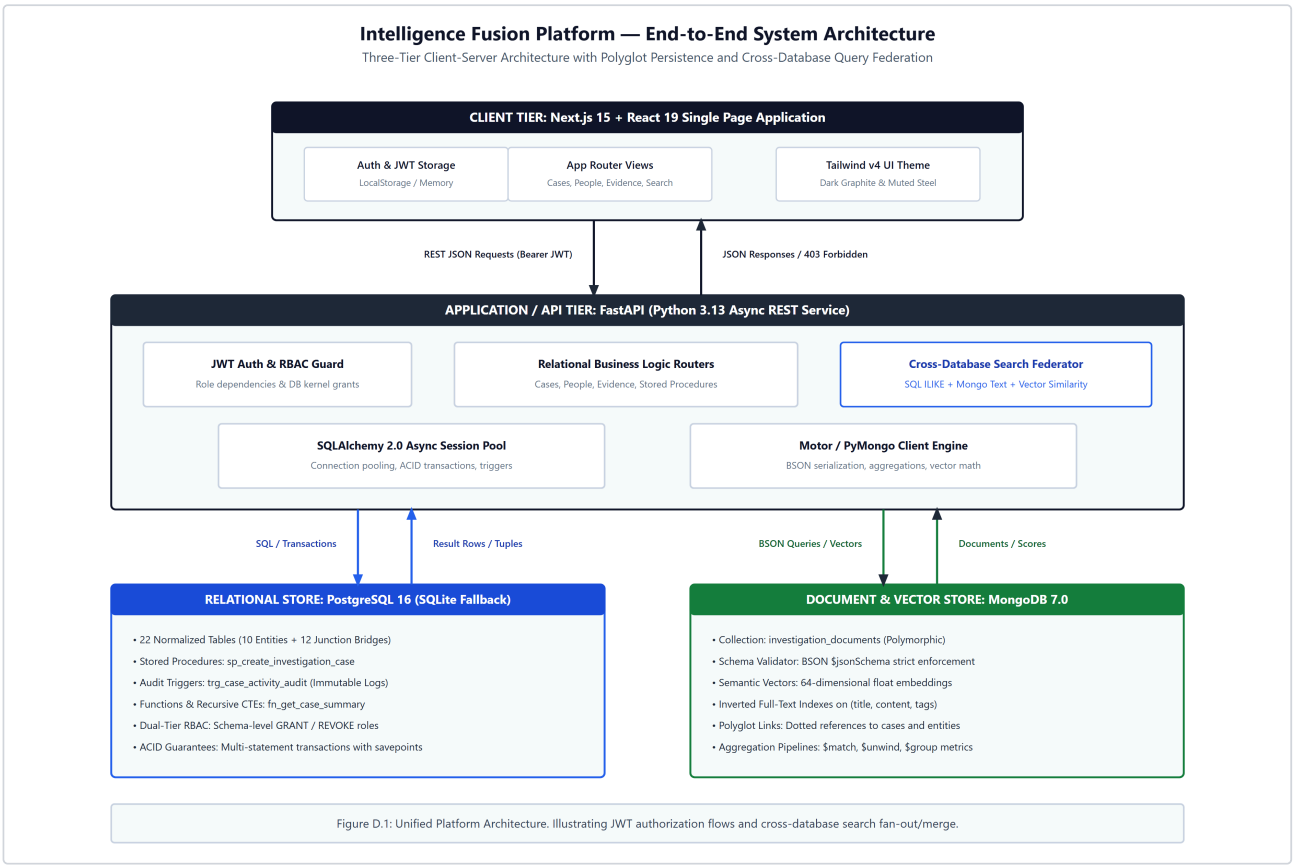


Figure 1: *System Architecture & Data Flow. Demonstrates three-tier client-server structure, JWT authorization flow, and cross-database search federation.*

2. REST API Endpoint Directory

Table 1 catalogs the platform's primary REST endpoints, detailing their HTTP method, route, access control requirements, storage tier, and operation.

Method	API Route	Required Role	Storage Store	Operational Description
`POST`	`/api/auth/login`	Public (Unrestricted)	PostgreSQL (officers)	Verifies bcrypt password; issues signed HS256 JWT.
`GET`	`/api/cases`	Authenticated	PostgreSQL (cases)	Lists case dossiers with priority/status filtering.
`POST`	`/api/cases`	`Admin, Investigator`	PostgreSQL (SP)	Invokes `sp_create_investigation_case` + audit trigger.
`GET`	`/api/cases/{id}`	Authenticated	PostgreSQL (Function)	Returns case summary via `fn_get_case_summary`.
`GET`	`/api/evidence`	Authenticated	PostgreSQL (evidence)	Lists seized evidence items and current locations.
`POST`	`/api/evidence/{id}/custody`	`Admin, Evidence Officer`	PostgreSQL (Tx)	Multi-statement custody transfer with SHA-256 hash.
`GET`	`/api/documents`	Authenticated	MongoDB (docs)	Retrieves polymorphic intelligence documents.
`GET`	`/api/search`	Authenticated	Polyglot (SQL + Mongo)	Parallel fan-out across SQL ILIKE and Mongo text index.
`GET`	`/api/search/semantic`	Authenticated	MongoDB + Vector	64-dimensional dense semantic vector similarity query.
`GET`	`/api/graph/relationships`	Authenticated	Polyglot Graph	Traverses 12 junction bridges for entity graph view.
`POST`	`/api/officers`	`ADMIN ONLY`	PostgreSQL (officers)	Provisions officer credentials (enforces 403 on others).

3. Next.js 15 Frontend Implementation

The user interface is built using Next.js 15, React 19, and Tailwind CSS v4. In keeping with modern operational tool standards, we designed the interface with a dark graphite base palette (`#070A0F`, `#0B0F17`, `#0E1420`), muted steel blue accents, structured data tables, and an interactive entity relationship canvas powered by `@xyflow/react`.

- **Case Management View:** High-density sortable tables displaying active cases, priority tags, lead investigator assignments, and status filters.
- **Evidence Locker & Chain of Custody:** Displays seized physical/digital exhibits, custody transfer histories, and cryptographic SHA-256 validation status.
- **Federated & Semantic Search:** Multi-mode search interface supporting SQL substring matching, MongoDB full-text keyword searches, and dense vector similarity queries.
- **Entity Graph Visualization:** Interactive node-link canvas rendering multi-degree suspect networks, linked vehicles, communication devices, and organizational hierarchies.

4. Originality & Architectural Extensions

To demonstrate mastery beyond baseline course expectations, we implemented five significant architectural extensions:

1. **Deterministic 64-Dimensional Semantic Vector Search:** We developed a custom offline vector search engine (`backend/app/core/vector.py`) that converts queries into 64-dimensional float embeddings and computes cosine similarity against stored documents. This allows semantic intelligence discovery without relying on third-party cloud APIs.
2. **Dual-Tier Role-Based Security:** We implemented defense-in-depth authorization by enforcing permissions at both the application layer (FastAPI JWT claims) and the database engine layer (PostgreSQL roles with table-level `GRANT` / `REVOKE`).
3. **Cross-Database Polyglot Entity Resolution:** The search router dispatches queries to PostgreSQL and MongoDB in parallel using `asyncio.gather`, merging relational records, polymorphic documents, and vector similarity ranks into a unified response payload.
4. **Deterministic Dual-Database Engine Fallback:** To ensure the application runs reliably in any evaluation environment, the database engine automatically detects PostgreSQL availability and automatically falls back to an embedded SQLite database if PostgreSQL is offline.
5. **Automated Audit Triggers & Stored Procedures:** Case provisioning and status changes trigger automated, immutable audit log insertions directly inside the database kernel (`trg_case_activity_audit`), ensuring an unalterable forensic record.

5. Team Contributions & Division of Responsibilities

Both team members contributed actively to the architecture, implementation, and verification of the project:

Team Member	Primary Focus	Core Deliverables & Technical Responsibilities
Saatvik Gupta (`PES1UG24CS395`)	Relational Architecture & Backend Systems	Designed the 22-table relational schema; authored `schema.sql`, DDL/DML scripts, stored procedure `sp_create_investigation_case`, function `fn_get_case_summary`, and trigger `trg_case_activity_audit`; implemented FastAPI REST services, dual-tier RBAC, and the 64-dimensional vector search engine.
Satyam Goyal (`PES1UG24CS424`)	NoSQL Modeling & Frontend User Interface	Designed the MongoDB document model, BSON schema validator, and aggregation pipelines; authored Next.js 15 views, the Tailwind v4 graphite design system, the interactive `@xyflow/react` relationship graph, and client-side API state caching.

6. Official Evaluation Matrix Compliance Map

Table 2 maps all 14 components of the official university evaluation rubric to the corresponding submission documents, verifying complete coverage across the platform.

#	Official Rubric Component	Marks	Submission Deliverable	Repository Evidence Path
1	ER Diagram	2	`01_ER_Diagram.pdf` (pp. 1–5)	`docs/diagrams/chen_*.png`, `crowsfoot_*.png`
2	Relational Schema with Keys & Constraints	3	`02_Relational_Schema.pdf` (pp. 1–3)	`database/sql/schema.sql`, `docs/database-design.md`
3	SQL Database Implementation (DDL)	4	`03_SQL_DDL_Implementation.pdf` (pp. 1–3)	`database/sql/schema.sql`
4	NoSQL / Vector Database Creation	4	`04_NoSQL_Vector_Database_Creation.pdf` (pp. 1–3)	`database/mongodb/collection-schema.json`
5	Technology Justification	1	`05_Technology_Justification.pdf` (pp. 1–2)	`docs/technology-justification.md`
6	DML Operations & Transactions	4	`06_DML_Operations.pdf` (pp. 1–3)	`database/sql/dml-examples.sql`
7	Nested, Correlated, Recursive & JOIN Queries	4	`07_Queries_Nested_Correlated_Recursive_Joins.pdf`	`database/sql/advanced-queries.sql`
8	Advanced SQL (Procedure, Trigger, Function, CTE)	6	`08_Advanced_SQL_Procedure_Function_Trigger_CTE.pdf`	`database/sql/advanced-concepts.sql`
9	NoSQL / Vector Features & RUD Operations	7	`09_NoSQL_Vector_Features_and_RUD.pdf`	`database/mongodb/rud-examples.js`, `vector.py`
10	Next.js User Interface	3	`11_Project_Report.pdf` (Section 3)	`frontend/src/views/*`
11	Backend API & Database Integration	2	`11_Project_Report.pdf` (Section 1–2)	`backend/app/routers/*`
12	Authentication & Dual-Tier RBAC in Database	4	`10_Authentication_and_RBAC.pdf`	`database/sql/roles-and-grants.sql`, `auth.py`
13	Documentation & Git Quality	2	`11_Project_Report.pdf` (Section 1–6)	`README.md`, Clean Git History, `start.ps1`
14	Originality, Contribution & Modifications	4	`11_Project_Report.pdf` (Section 4–5)	`docs/modifications.md` (5 major extensions)
Total	**14 Components (Official Evaluation Rubric)**	**50**	**Complete 11 PDF Deliverable Suite**	**100% Repository Compliance Verified**